PROBLEMA:	**OAT 3 - Persistindo os meus Dados**	**DATA:**	**09/10/2026**
PILOTO (A):	Italo Mendes	**COPILOTO (A):**	Áleffy de Almeida
QA:	Augusto Brandão	**ARQUITETO (A):**	Eduardo Almeida
SCRUM MASTER:	Hellen Verena		

Objetivo: Estabelecer a conectividade com o banco de dados MySQL e configurar a infraestrutura básica via JDBC. **Fatos (O Scrum Master lê as perguntas. A equipe busca a resposta exata na OAT 3 e o Arquiteto documenta no repositório):**

- **Banco de Dados Oficial e ORM:** Qual é o banco de dados oficial e qual é a restrição tecnológica absoluta sobre ORMs (como JPA/Hibernate)?
    - *Fato extraído: A OAT 3 exige MySQL e JDBC nativo, com SQL manual; JPA, Hibernate e outras APIs ORM do Spring estão proibidos.*
- **Versionamento do Banco:** O que a equipe deve criar e versionar junto com o código-fonte no repositório?
    - *Fato extraído: O script SQL com DDL de criação das tabelas deve ser versionado junto ao código, com DML inicial somente se houver dados de partida. No projeto: sql/mecaniqa.sql.*

Brainstorm Investigativo (Questões e Ideias):

- **A Conexão:** Como implementaremos a conexão segura com o banco de dados?
    - Resposta: A classe ConexaoMySql usa DriverManager.getConnection e lê DB_URL, DB_USER e DB_PASSWORD das variáveis de ambiente. A senha não fica no repositório.

- **Padrão de Repositório:** A OAT 3 exige uma interface genérica de repositório. Como adaptaremos os repositórios para implementar esta nova interface baseando-se em JDBC?
    - Resposta: PecaRepository, ServicoRepository, OrdemServicoRepository e PedidoPecasRepository implementam Repository<T, ID> com PreparedStatement e SQL manual.

Cenários de Teste (Liderado pelo QA):

- Validar se a aplicação consegue iniciar a conexão com o banco de dados e lançar um erro claro caso as credenciais do banco estejam erradas.
    - Registro do QA: A API iniciou e acessou o MySQL autenticado. O relatório não comprova teste específico com credenciais incorretas.

Entrega do Encontro:

- **Artefatos realizados: o script de criação das cinco tabelas MySQL foi versionado em sql/mecaniqa.sql. Não foram inseridos dados iniciais.**
- **Código realizado: a classe ConexaoMySql faz a conexão JDBC e os repositórios utilizam PreparedStatement com SQL manual.**

PROBLEMA:	**OAT 3 - Persistindo os meus Dados**	**DATA:**	**16/10/2026**
PILOTO (A):	Hellen Verena	**COPILOTO (A):**	Italo Mendes
QA:	Áleffy de Almeida	**ARQUITETO (A):**	Augusto Brandão
SCRUM MASTER:	Eduardo Almeida		

Objetivo: Criar as rotinas de mapeamento manual e lidar de forma adequada com regras de negócio e validações criando exceções customizadas sem @ExceptionHandler. **Fatos (O Scrum Master lê as perguntas. A equipe busca a resposta exata na OAT 3 e o Arquiteto documenta no repositório):**

- **Conversão de Dados:** O que os estudantes devem criar para lidar com os ResultSets trazidos pela API JDBC?
  - *Fato extraído: Criar Mappers de ResultSet para converter registros JDBC em objetos Java. Foram implementados os Mappers de Peça, Serviço, Ordem de Serviço e Pedido de Peças.*
- **Restrição de Tratamento:** Qual anotação global fornecida pelo Spring Boot está expressamente proibida de ser utilizada e como as exceções devem ser tratadas?
  - *Fato extraído: @ExceptionHandler global é proibido; o tratamento deve ser manual, por try/catch nos Controllers ou Services.*

Brainstorm Investigativo (Questões e Ideias):

- **Mapeamento:** Onde as classes Mappers vão agir para não poluir o Controller e manter os dados íntegros?
    - Resposta: Os ResultSet Mappers atuam na camada de acesso aos dados, convertendo as linhas SQL em entidades Java; os Controllers recebem e retornam DTOs por meio dos Services.

- **A Visão do Erro:** Quais exceções vamos criar e em que ponto exato elas serão disparadas e capturadas no nosso try-catch manual?
    - Resposta: DadosInvalidosException é lançada em validações dos Services e RecursoNaoEncontradoException quando um registro não existe; os Controllers capturam com try/catch e retornam 400 ou 404.

Cenários de Teste (Liderado pelo QA):

- Testar a requisição de busca para um ID inexistente; o QA valida se o sistema retorna erro 404 de forma manual e segura via a exceção customizada.
    - Registro do QA: Foram confirmadas respostas HTTP 404 ao consultar registros inexistentes ou removidos; os Controllers tratam RecursoNaoEncontradoException manualmente.

Entrega do Encontro:

- **Código realizado: foram criados os quatro ResultSet Mappers e as exceções DadosInvalidosException e RecursoNaoEncontradoException, tratadas nos Controllers por try/catch.**

PROBLEMA:	**OAT 3 - Persistindo os meus Dados**	**DATA:**	**23/10/2026**
PILOTO (A):	Eduardo Almeida	**COPILOTO (A):**	Hellen Verena
QA:	Italo Mendes	**ARQUITETO (A):**	Áleffy de Almeida
SCRUM MASTER:	Augusto Brandão		

Objetivo: Aplicar o padrão de projeto comportamental *Template Method* para padronizar as operações de acesso ao banco de dados e evitar repetição de código *boilerplate*.
Fatos (O Scrum Master lê as perguntas. A equipe busca a resposta exata na OAT 3 e o Arquiteto documenta no repositório):

- **Repetição de Código:** Qual padrão comportamental a OAT exige para padronizar o ciclo de vida das operações do JDBC (abrir, preparar, executar e fechar)?
    - *Fato extraído (conforme o caderno desta sessão): criar classe base abstrata com Template Method para o ciclo do JDBC. Pendência: essa classe não está implementada na versão atual do projeto.*

Brainstorm Investigativo (Questões e Ideias):

- **Esqueleto do Algoritmo:** Como estruturar a classe base abstrata para que o bloco de abertura, try, catch e finally não se repita em todo método de CRUD?
    - Resposta: A proposta da sessão é centralizar abrir/preparar/executar/fechar numa classe base abstrata. O código entregue ainda usa try-with-resources em cada Repository; o Template Method não foi implementado.
- **Delegação:** Quais partes da rotina serão obrigatórias que as subclasses implementem?
    - Resposta: Na proposta do Template Method, as subclasses prepariam os parâmetros da consulta e converteriam o ResultSet; a classe base cuidaria do ciclo da conexão. Isso ainda não consta no código.

Cenários de Teste (Liderado pelo QA):

- Garantir que as conexões sempre estão sendo encerradas independentemente de a query falhar ou obter sucesso.
    - Registro do QA: O código utiliza try-with-resources para fechar Connection, PreparedStatement e ResultSet. Não foi apresentado teste isolado de falha de consulta para verificar o fechamento.

Entrega do Encontro:

- **Situação do código: a classe abstrata de Template Method não foi implementada. Os repositórios atuais usam JDBC e try-with-resources para fechar os recursos.**

PROBLEMA:	**OAT 3 - Persistindo os meus Dados**	**DATA:**	**24/10/2026**
PILOTO (A):	Augusto Brandão	**COPILOTO (A):**	Eduardo Almeida
QA:	Hellen Verena	**ARQUITETO (A):**	Italo Mendes
SCRUM MASTER:	Áleffy de Almeida		

Objetivo: Finalizar a integração e migração do sistema para que todas as entidades completem suas operações usando a nova persistência. **Fatos (O Scrum Master lê as perguntas. A equipe busca a resposta exata na OAT 3 e o Arquiteto documenta no repositório):**

- **Requisitos de Negócio:** O que deve acontecer com os CRUDs de Peça, Serviço, OS e Pedido e qual a expectativa para o formato de JSON recebido e devolvido pelo cliente?
    - *Fato extraído: Manter as operações existentes com persistência MySQL e preservar o formato dos JSONs recebidos e devolvidos pela API REST.*
- **Versionamento Final:** Qual branch será avaliada com todo o código integrado e o script do banco até a data limite?
    - *Fato extraído: O professor avaliará somente a branch main no prazo previsto. Situação conferida: o desenvolvimento está em PRs e ainda aguarda integração à main.*

Brainstorm Investigativo (Questões e Ideias):

- **Apresentação Final:** Como a equipe vai estruturar o *storytelling* para demonstrar o Padrão *Template Method* operando na camada de banco de dados para o professor?
    - Resposta: A apresentação deverá explicar a migração para MySQL, JDBC, Repository, Mappers, Services e exceções. Não demonstrar Template Method como implementado enquanto essa classe abstrata não existir.

Cenários de Teste (Liderado pelo QA):

- Executar as requisições (via Collection) e validar no MySQL se a inserção e relacionamentos foram cadastrados corretamente nas tabelas.
    - Registro do QA: Os fluxos principais passaram nos testes funcionais com MySQL, incluindo persistência após reinício e item repetido. Não há evidência da execução integral da collection no Postman.

Entrega do Encontro:

- **Situação do código: os fluxos implementados passaram nos testes funcionais com MySQL, inclusive após reiniciar a API. Os commits estão nos PRs #13, #14 e #15; ainda não houve integração à main. Apresentação pendente.**